

Spatial index such as an octree/KD tree

Terrain elevation -> dem / dtm / terrain mesh

Point Cloud View — Final Context

What the user sees →

- Full-screen **3D point cloud** of the reconstructed survey area.
- Dense metric 3D points representing the actual reconstructed environment.
- Point cloud is georeferenced and aligned with real-world coordinates/elevation.
- Each point can contain:
 - Point ID
 - X, Y, Z
 - Latitude/Longitude
 - Terrain elevation
 - RGB/color
 - Semantic class
 - Reconstruction confidence
- Camera/flight trajectory can optionally be displayed.
- This is primarily the **raw/technical geometric representation**; detailed user analysis is mainly performed through Semantic and Cesium views.

What the user can do →

- Rotate, pan, zoom.
- Select **1, 2, 3, 4, or 5+ points**.
- Change point size/density.
- Filter by elevation.
- Filter by semantic class.
- Filter by confidence.
- Change visualization mode: RGB / Semantic / Elevation / Confidence.
- Show/hide flight trajectory.

- Clear and restart point selection.
- Switch to Cesium or Semantic View while preserving the selected points.

What happens when they click / select something →

Every selected point is immediately resolved from the **pre-stored point-cloud data**.

The system retrieves:

- Point ID
- X, Y, Z
- Latitude/Longitude
- Elevation of the terrain underneath that point
- Semantic class
- Associated Object ID
- Confidence

The selected point is stored in the current session.

If multiple points are selected, the system calculates the required measurements from the already available metric data rather than running the ML model again.

Measurement & Inspection workflow →

1 Point

Show:

- X, Y, Z coordinates
- Terrain elevation at the point

2 Points

For both points:

- X, Y, Z coordinates
- Terrain elevation

Between the two points:

- 3D distance
- Horizontal distance
- Vertical difference
- Slope

Terrain statistics:

- Maximum elevation
- Minimum elevation
- Average elevation
- Elevation range

3 Points

For all three points:

- X, Y, Z coordinates
- Terrain elevation

Between every pair:

- 3D distance
- Horizontal distance
- Vertical difference
- Slope

Triangle analysis:

- Triangle area
- Perimeter
- 3D side lengths
- Slope of the plane

Terrain statistics:

- Maximum elevation
- Minimum elevation
- Average elevation
- Elevation range

4 Points

Show:

- X, Y, Z coordinates
- Terrain elevation

Analysis:

- Boundary perimeter
- Surface area
- Elevation statistics

- Volume **only when a valid base/reference surface or closed geometry is defined**

Terrain statistics:

- Maximum elevation
- Minimum elevation
- Average elevation
- Elevation range

5+ Points

Show only:

- X, Y, Z coordinates
- Terrain elevation
- Maximum elevation
- Minimum elevation
- Average elevation
- Elevation range

No additional distance/area/perimeter/volume calculations should be shown.

Layer / Filter controls →

- Point size.
 - Point density.
 - Elevation range.
 - Semantic class.
 - Confidence range.
 - RGB rendering.
 - Semantic rendering.
 - Elevation rendering.
 - Confidence rendering.
 - Camera/flight trajectory.
 - Survey boundary.
-

How switching between modes works →

- Selected points remain preserved when switching from Point Cloud → Semantic or Cesium.
- No reconstruction or ML processing happens during switching.

- The frontend loads the already-generated visualization assets and uses the stored spatial/object relationships.

Point Cloud → Semantic

- Objects containing the selected points become highlighted with increased opacity.
- Their normal semantic colors are retained.

Point Cloud → Cesium

- Only the selected point/location is displayed.
- The containing object is **not highlighted**.
- No additional object emphasis is applied.

How the three modes stay synchronized →

The backend maintains a common spatial relationship:

Point ID → XYZ → Geographic Location → Object ID → Semantic Class → Cesium Geometry

Therefore:

Point Cloud selection

- corresponding Object ID
- Semantic object identified
- Cesium geographic location identified.

The three views are different representations of the **same georeferenced reconstruction**, not separately generated models.

Is this possible? How? Accuracy + processing →

Yes.

Best accuracy-first approach:

AMB3R-SfM

- metric depth/camera poses
- dense point-cloud fusion
- semantic projection
- terrain/object reconstruction

- GPS/telemetry alignment
- WGS84/ECEF/ENU
- Umeyama transformation
- georeferenced point cloud + mesh.

For fast point selection, use a **spatial index such as an octree/KD-tree**.

For terrain elevation, generate and store a dedicated **DEM/DTM/terrain mesh** so the backend can instantly query terrain elevation beneath any selected point.

1 m accuracy

≤ 1 m should be treated as the target and can be achievable, but cannot be guaranteed from ordinary GPS + RGB video alone.

Accuracy requires:

- Metric reconstruction.
- Accurate camera calibration.
- Good GPS quality.
- Accurate timestamp synchronization.
- GPS-camera alignment.
- Robust georeferencing.
- Independent checkpoints/GCPs or RTK/PPK where available.
- RMSE/median/95th-percentile validation.
- Confidence-based filtering.

The system should report the **measured accuracy**, rather than simply claiming ≤ 1 m.

<15 minute processing

A <15-minute target for a 10-minute video is achievable as an engineering target, but requires benchmarking.

Use:

- Adaptive keyframes.
 - GPU processing.
 - Parallel preprocessing.
 - AMB3R-SfM on selected frames.
 - Efficient point-cloud fusion.
 - Selective VDA refinement only for weak regions.
 - Parallel asset generation.
 - Precomputed spatial indexes and measurements.
-

Requirement in model →

The **Point Cloud must be generated as a measurement-ready, georeferenced dataset**, not merely as a collection of rendered points.

Point-level data

Each point should store or be queryable for:

- Persistent Point ID
- X, Y, Z in metric coordinates
- Latitude
- Longitude
- Point elevation/Z
- **Terrain elevation at the point's XY position**
- RGB/color
- Semantic class
- Semantic confidence
- Reconstruction confidence
- Object ID where applicable

Terrain data

A separate terrain representation must be generated:

- DTM/DEM or terrain mesh
- Terrain elevation lookup
- Terrain surface geometry
- Elevation statistics
- Slope information where required

This is necessary because:

Point elevation ≠ terrain elevation

For example, a roof point may be at 180 m while the terrain underneath is 165 m.

Object association

Maintain:

Point ID → Object ID → Semantic Class → Object Geometry

So if selected points belong to a building, the system can immediately identify that building in Semantic View.

Spatial indexing

The point cloud must be indexed for fast:

- Point selection
- Nearest-point lookup
- Region queries
- Object association
- Terrain queries
- Cross-view synchronization

Georeferencing metadata

Store:

- CRS
 - Units
 - WGS84 reference
 - ECEF/ENU transformation
 - Scale
 - Rotation
 - Translation
 - Model origin/reference
 - GPS-camera alignment parameters
 - Accuracy/QA metrics
-

Backend & model flow during loading time for this view →

Point Cloud should be the first major reconstruction asset generated by the ML pipeline.

During the user's ~15-minute processing period:

Video + GPS + Flight Metadata

- Video/GPS preprocessing
- Adaptive keyframe selection
- **AMB3R-SfM**
- Camera poses + metric depth + pointmaps
- Pointmap/point-cloud fusion
- Outlier/confidence filtering
- GPS/telemetry alignment
- WGS84/ECEF/ENU + Umeyama georeferencing
- **Final georeferenced point cloud**

Then the backend immediately prepares and stores:

Point Cloud Data

- XYZ
- Lat/Lon
- Point elevation
- Terrain elevation
- RGB
- Semantic class
- Confidence
- Point ID
- Object ID

Derived Data

- DEM/DTM/terrain mesh
- Spatial index
- Object-point relationships
- Geographic bounds
- Measurement-ready geometry
- QA/accuracy metrics

Then, while the remaining pipeline continues, the backend can generate the other visualization assets:

Point Cloud

- Semantic 3D
- Georeferenced Cesium model/3D Tiles
- Cross-view object mapping
- Final metadata.

By the time the reconstruction reaches **100%**, all three visualization modes and their relationships should already be prepared.

Therefore, when the user enters visualization:

**Point Cloud loads immediately → Semantic View loads immediately →
Cesium View loads immediately → switching between them requires no
ML/reconstruction processing.**

The frontend is essentially **reading precomputed assets and indexed data**, while the expensive ML work happens only once in the backend reconstruction job.